

09

강

R 컴퓨팅

R 프로그래밍의 핵심 구조 (2)

통계·데이터과학과 장영재 교수

학습목차

1 프로그래밍의 기본 구조

2 R 함수 만들기

3 실습하기

학습목표

- 1 조건문과 반복문 등 제어문의 구조를 이해하고 활용할 수 있다.
- 2 함수의 개념과 구조를 이해할 수 있다.
- 3 사용자가 직접 함수를 작성하고 사용할 수 있다.



01

프로그래밍의 기본 구조

1. 프로그래밍 기본 구조

▶ 조건문

- 조건문은 특정한 조건을 만족했을 경우에만 프로그램 코드를 수행하는 제어 구문을 의미
- 조건문에는 항상 논리 연산이 수반되며, 조건문의 구체적인 표현식은 조건의 개수, 조건문의 위치, 조건에 따른 명령수행 방식 등에 따라 구분

1) if(조건) 실행문

if 조건문은 괄호 안의 조건이 참이면 실행문을 수행하는 표현식으로,
실행문이 여러 개일 경우 실행문을 중괄호({ })로 묶어서 사용

1. 프로그래밍 기본 구조

예제 5-5 일정한 기준을 만족할 때만 명령을 수행하는 if 조건문

```
> x <- c(1,2,3,4)
```

```
> y <- c(3,4,5,6)
```

```
> if(sum(x) < sum(y)) print(x) # x의 합이 y의 합보다 작을 때 x를 출력
```

```
[1] 1 2 3 4
```

```
> if(sum(x) < sum(y)) {
```

```
+ print(mean(x))
```

```
+ print(var(x))
```

```
+ } # x의 합이 y의 합보다 작으면 x의 평균과 분산을 출력
```

```
[1] 2.5
```

```
[1] 1.66667
```

1. 프로그래밍 기본 구조

2) if (조건) else 조건 실행문

if (조건) else 조건 실행문은 괄호 안의 조건이 참이면 참일 때의 실행문을 수행하고,
거짓일 때는 참이 아닐 때의 실행문을 수행하는 표현식

예제 5-6 [예제 5-5]에서 사용한 동일한 벡터 x, y를 정의하고, 평균값을 비교하여 "Mean(x) > Mean(y)"

또는 "Mean(x) < Mean(y)" 등으로 출력하는 조건문. 또한 x의 평균이 y의 평균보다 크면

x의 평균과 분산을 출력하고, 그렇지 않으면 y의 평균과 분산을 출력

```
> x <- c(1,2,3,4)
```

```
> y <- c(3,4,5,6)
```

```
> if(mean(x)>mean(y)) print("Mean(x)>Mean(y)") else
```

```
+ print("Mean(x)<Mean(y)") # x의 평균이 y의 평균보다 크면
```

```
# "Mean(x)>Mean(y)" 출력
```

```
# x의 평균이 y의 평균보다 작으면
```

```
# "Mean(x)<Mean(y)" 출력
```

```
[1] "Mean(x)<Mean(y)"
```

1. 프로그래밍 기본 구조

```
> if(mean(x)>mean(y)) { # x의 평균이 y의 평균보다 크면  
+ print(mean(x))  
+ print(var(x)) # x의 평균과 분산을 출력  
+ } else { # 그렇지 않으면  
+ print(mean(y))  
+ print(var(y)) # y의 평균과 분산을 출력  
+ } # if와 else의 조건문들이 중괄호로 묶여 있음  
[1] 4.5  
[1] 1.666667
```


1. 프로그래밍 기본 구조

3) 중첩 조건문

중첩 조건문은 조건문 내에 조건문이 위치하는 형태

예제 5-7 벡터 score를 정의하고, 평균 점수를 기준으로 다음과 같이 등급을 판별 : 평균이 80점 이상이면 "합격"을 출력하고
90점 이상이면 "우수"를 추가로 출력하며 평균이 80점 미만이면 " 불합격 " 을 출력

```
> score <- c(85, 90, 88, 92, 98)
```

```
> avg <- mean(score)
```

```
> # 중첩 조건문
```

```
> if (avg >= 80) {
```

```
+ cat("합격\n")
```

```
+ if (avg >= 90) {
```

```
+ cat("우수\n")
```

```
+ }
```

```
+ } else {
```

```
+ cat("불합격\n")
```

```
+ }
```

```
합격
```

```
우수
```

1. 프로그래밍 기본 구조

4) ifelse 조건문

ifelse 조건문은 조건, 조건이 참일 때의 결과, 조건이 거짓일 때의 결과를 한 줄로 표현할 수 있는 함수

예제 5-8 벡터 temperature를 정의하고, 각 값이 0도 이상이면 "above freezing", 0도 미만이면

"below freezing"을 출력하도록 ifelse() 함수를 사용. 또한 평균 온도가 25도 이상이면 "Hot",

10도 이상 25도 미만이면 "Mild", 10도 미만이면 "Cold"를 출력

```
> temperature <- c(-3, 0, 12, 18, 27, 5)
```

```
>
```

```
> # 각 온도별 상태 판별
```

```
> status <- ifelse(temperature >= 0, "above freezing", "below freezing")
```

```
> print(status)
```

```
[1] "below freezing" "above freezing" "above freezing" "above freezing"
```

```
[5] "above freezing" "above freezing"
```

```
>
```

1. 프로그래밍 기본 구조

> # 평균 온도 계산

> avg_temp <- mean(temperature)

>

> # 평균에 따른 분류

> ifelse(avg_temp >= 25, "Hot",

+ ifelse(avg_temp >= 10, "Mild", "Cold"))

[1] "Cold"

>

1. 프로그래밍 기본 구조

5) switch문

switch문은 switch(매개변수, 실행문1, 실행문2, ...)와 같이 사용하며, 매개변수의 값에 따라 조건에 맞는 실행문을 찾아 수행

예제 5-9 switch의 매개변수값이 문자열을 갖는 경우

```
> x <- c(1,2,3,4)
> type <- "var"
> switch(type, mean=mean(x), median=median(x), sum=sum(x), var=var(x))
[1] 1.66667
```

예제 5-10 switch의 매개변수값이 정수값을 갖는 경우

```
> x <- c(1,2,3,4)
> switch(-1, mean(x), sum(x), var(x))
> switch(0, mean(x), sum(x), var(x))
> switch(1, mean(x), sum(x), var(x))
[1] 2.5
```

switch문에서 매개변수값이 정수값을 갖게 되면 이 정수값에 해당하는 순서의 실행문을 수행

1. 프로그래밍 기본 구조

▶ 반복문

1) for 반복문

for 반복문은 for(변수 in 반복횟수) {실행문}과 같은 형태로서 중괄호 내에 위치한 실행문을 반복횟수만큼 실행하는 구문

예제 5-11 for 반복문을 실행하여 다음과 같은 결과를 출력

```
[1] 1
```

```
[1] 2 2
```

```
[1] 3 3 3
```

```
[1] 4 4 4 4
```

```
[1] 5 5 5 5 5
```

```
> for(i in 1:5) print(rep(i,i))
```

예제 5-12 for 반복문을 실행하여 1에서 10까지의 합 구하기

```
> x <- 0
```

```
> for(i in 1:10)
```

```
+ x <- x+i
```

```
> x
```

```
[1] 55
```

1. 프로그래밍 기본 구조

2) while 반복문

while 반복문은 while(조건) {실행문}의 형태로 쓰이며 조건이 참일 때까지 실행문을 수행하는 반복문

예제 5-13 while 반복문을 실행하여 [예제 5-11]에서 제시된 것과 동일한 결과를 출력

```
> i <- 1
> while(i<=5) {
+ print(rep(i,i))
+ i <- i+1          # 1씩 증가시켜 i 자신에게 할당
+ }
```

예제 5-14 while 반복문을 실행하여 1에서 10까지의 합 구하기

```
> i <- 1
> x <- 0
> while(i <= 10) {
+ x <- x+i
+ i <- i+1
+ }
> x
[1] 55
```

1. 프로그래밍 기본 구조

3) repeat 반복문

repeat 반복문은 repeat {실행문}과 같은 형태로 사용되며 실행문을 계속 수행하다가 break를 만나면 반복수행을 멈추고 작업을 종료

예제 5-15 repeat 반복문을 실행하여 [예제 5-11]에서 얻었던 결과를 똑같이 출력하기

```
> i <- 1
> repeat {
+ if(i > 5) break
+ print(rep(i,i))
+ i <- i+1
+ }
```

예제 5-16 repeat 반복문을 실행하여 1에서 10까지의 합 구하기

```
> i <- 1
> x <- 0
> repeat {
+ if(i > 10) break
+ x <- x+i
+ i <- i+1
+ }
> x
[1] 55
```

1. 프로그래밍 기본 구조

▶ 무조건 분기문

1) break 분기문

break 분기문은 루프를 탈출하는 제어문으로 중첩 루프가 사용되었을 경우, break가 위치한 해당 루프에서만 빠져나오게 된다는 점에 유의

예제 5-17 1에서 10까지 합을 구하는 루프에서 합이 25 이하가 되는 시점까지 각 단계별 합계를 출력

```
> x <- 0
```

```
> for(i in 1:10) {
```

```
+ x <- x+i
```

```
+ if(x > 25) break # 만약 각 단계까지의 합계 x가 25보다 크다면
```

```
                  # break문을 수행하여 반복문을 벗어나게 됨
```

```
+ print(x)
```

```
+ }
```

```
[1] 1
```

```
[1] 3
```

```
[1] 6
```

```
[1] 10
```

```
[1] 15
```

```
[1] 21
```


1. 프로그래밍 기본 구조

예제 5-18 for 반복문을 중첩반복문 형태로 사용하여 다음과 같은 결과를 출력해 보기

결과 내용

[1] 1 * 1 = 1

[1] 2 * 1 = 2

[1] 3 * 1 = 3

for 반복문

> for(i in 1:9) {

+ if(i > 3) break

+ for(j in 1:9) {

+ if(j > 1) break

+ cat(i, "*", j, "=", i*j, "\n")

+ }

+ }

1. 프로그래밍 기본 구조

2) next 분기문

next는 break보다 더 강제적인 제어문으로서 반복문을 수행하더라도 조건을 충족하면 next 이후의 실행문을 수행하지 않고 계속 건너뛰게 됨

예제 5-19 while 반복문과 next 분기문을 실행하여 1에서 10까지 변수값을 증가시키되 8보다 작을 때까지는 값을 출력하지 않고 8 이상인 값과 이들의 합을 출력

```
> i <- 1
> x <- 0
> while(i < 10) {
+ i <- i+1
+ if(i < 8) next # next 이후의 명령은 if문의 조건이 충족되는 한 미수행
+ print(i)
+ x <- x+i
+ }
[1] 8
[1] 9
[1] 10
> x
[1] 27
```

02

R 함수 만들기

2. R 함수 만들기

- ▶ 함수란 특정한 작업을 독립적으로 수행하는 프로그램 코드의 집합체
 - R의 내장함수에 사용자가 원하는 특정한 기능이 구현되어 있지 않다면 사용자가 직접 함수를 생성하여 원하는 기능을 수행할 수 있음
- ▶ 함수의 장점
 - 작업을 작은 단위로 분할하여 수행하도록 함으로써 효율성 제고
 - 자유로운 수정·보완이 가능하므로 작업의 유연성이 확보됨
 - 작업에 사용하는 스크립트의 오류 등의 발생 원인 파악이 용이
 - 스크립트의 크기를 줄임으로써 작업 프로세스 이해가 용이
 - 사용자가 정의한 함수를 자유롭게 활용하여 작업 목적을 충족

2. R 함수 만들기

▶ 함수의 구조 및 생성방법

```
함수이름 <- function(매개변수) {함수의 몸체}
```

- function은 함수의 정의를 위해 사용하는 R의 내장함수
- list() 함수가 list 클래스 객체를 생성하듯이 function()이 함수 클래스 객체를 생성
 - * 함수는 매개변수와 몸체 부분으로 나눌 수 있음
- 매개변수는 일반적으로 함수 선언부나 정의 부분에 있는 변수를 의미
- 중괄호로 묶여 있는 함수의 몸체 부분은 함수가 호출될 때 실행하게 될 내용들

▶ 함수실행의 3단계

- 함수는 입력, 연산, 출력의 3단계를 거쳐 실행됨
- 입력단계에서는 매개변수를 통해 입력된 자료값을 읽어 들이는 작업을 수행
- 연산단계에서는 입력 데이터를 변환하거나 계산하는 작업을 실시
- 출력단계에서는 연산결과를 객체로 반환하거나 화면에 출력하는 작업을 실행

2. R 함수 만들기

▶ 변수의 종류

- C 등과 같은 프로그래밍 언어와 마찬가지로 R에서도 변수의 유효범위에 따라 지역변수, 전역변수, 매개변수(인수) 등으로 변수를 구분
- 전역변수는 어떤 변수의 영역 내에서도 접근할 수 있는 변수로, 프로그램이 종료될 때까지 사용 가능한 상태로 존재하다가 프로그램이 종료되면 소멸
- 지역변수란 전역변수와는 다르게 함수 내부에서만 사용이 가능한 변수로, 함수 내에서 정의된 후 함수가 호출되면 생성되었다가 함수가 종료되면 함께 소멸이 되는 변수
- 지역변수가 전역변수와 동일한 이름을 갖고 있을 경우에는 함수 내에서 지역변수처럼 인식되지만, 함수를 벗어나서는 전역변수 값이 그대로 보존되고 함수 내에서의 연산이 영향을 미치지 않음
- 매개변수는 함수로 전달되는 값으로, 함수를 생성할 때 소괄호 내에 선언해 주며 함수 내에서만 사용 가능

2. R 함수 만들기

예제 5-20 ① u, v 는 전역변수로서 함수 외부에서 정의하여 각각 1, 8 값을 할당, ② 매개변수 x 를 받아 x 에 1을 더하고 이러한 x 값에 u 의 값을 더한 뒤 다시 u 에 저장·출력하는 함수를 생성

```
> u <- 1      # 전역변수
> v <- 8      # 전역변수
> g <- function(x) {      # x: 매개변수
+ x <- x+1 # 전달된 인자에 1을 더함
+ u <- u+x  # 값을 할당받는 u는 함수 내부의 지역변수로 새로 정의
+ return(u)
+ }
> g(v)      # v에 8이라는 값이 할당되어 있으므로 g(8)의 의미
[1] 10
> u      # 전역변수는 값의 변화가 없음
[1] 1
> v      # 전역변수는 값의 변화가 없음
[1] 8
```

2. R 함수 만들기

▶ 함수의 생성방법

- R에서 함수를 생성하는 방법은 크게 세 가지로 나뉨
- R 콘솔에서 직접 입력하여 생성하는 방법과 fix 함수를 이용하는 방법, 그리고 외부 파일에 저장하여 읽어 들이는 방법

예제 5-21 ① R 콘솔에서 직접 입력하여 작성하는 방식으로 데이터 값을 받아 산술평균을 구하는 d.mean 함수를 작성,

② 간단한 예시 벡터를 정의하고 d.mean 함수를 이용하여 산술평균을 산출

```
> # 함수 정의
> d.mean <- function(x) {
+ s <- 0
+ for (i in 1:length(x)) {
+ s <- s + x[i]
+ }
+ return(s / length(x))
+ }
>
> # 예시 벡터
> x <- c(2, 4, 6, 8, 10)
```


2. R 함수 만들기

> # d.mean 함수로 평균 구하기

> d.mean(x)

[1] 6

>

> # 내장 mean() 함수와 비교

> mean(x)

[1] 6

2. R 함수 만들기

예제 5-22 fix() 함수를 이용하여 매개변수 data를 받아서 분산을 구하는 함수 만들기

R 콘솔에서 fix(함수이름)을 입력하면 함수이름과 함께 R Editor 창이 열리며, 여기서 함수를 작성하거나 수정

```
> d.var  
function(data) # 함수의 선언  
{  
+ data.var <- sum((data - d.mean(data))^2) / (length(data)-1) # 분산공식  
+ return(data.var) # data.var 값을 반환  
+ }
```

<함수의 생성과정>

2. R 함수 만들기

예제 5-23 데이터 값을 받아서 데이터의 최솟값과 최댓값을 각각 data.min, data.max 변수에 할당하고

data의 범위를 출력하는 함수 만들기 : 메모장을 열어서 다음과 같이 입력하고 rangefunction.txt로 저장

```
d.range <- function(data)      # 함수의 선언
+ {
+ data.min <- min(data)         # 데이터의 최솟값 산출
+ data.max <- max(data)         # 데이터의 최댓값 산출
+ c(data.min, data.max)         # 데이터의 범위를 출력
+ }
```

```
> source("C:\rangefunction.txt") # 함수의 생성
```

```
> d.range(x)                    # x의 범위 출력
```

```
> [1] -3.273532 8.275104
```

2. R 함수 만들기

▶ 함수의 편집

- 생성한 함수에 오류가 있다든지 작업을 효율적으로 수행하기 위해서 경우에 따라 함수를 편집할 필요가 있음
- fix() 함수나 edit() 함수를 사용하면 편집창이 열리게 되므로 이를 이용하여 함수를 편집

예제 5-24 ① x와 y라는 데이터 값을 받아서 두 수의 합을 계산하는 함수 f1과 차를 계산하는 f2를 생성,

② 함수 f1을 f라는 이름으로 지정하여 f(1,2)를 계산하고 f2를 f라는 이름으로 지정하여 f(1,2)를 계산하기

```
> f1 <- function(x,y) {return(x+y)}
```

```
> f2 <- function(x,y) {return(x-y)}
```

```
> f <- f1
```

f1을 f라는 이름으로 저장

```
> f(1,2)
```

```
[1] 3
```

```
> f <- f2
```

f2를 f라는 이름으로 저장

```
> f(1,2)
```

```
[1] -1
```

2. R 함수 만들기

예제 5-25 [예제 5-24]에서 생성한 f 1과 f 2 및 x와 y를 매개변수로 하는 함수 g를 생성하여 x와 y의 합과 차를 계산하는 작업을 수행

```
> g <- function(h,x,y) {h(x,y)}           # h는 함수를 받는 매개변수
```

```
> g(f1,1,2)                               # h는 f1, x=1, y=2
```

```
[1] 3
```

```
> g(f2,1,2)                               # h는 f2, x=1, y=2
```

```
[1] -1
```

2. R 함수 만들기

▶ 함수의 매개변수 지정

예제 5-26 x에 (10, 20, 30, 40)의 값을, y에 (1, 2, 3, 4)의 값을 저장하고 $x * y$ 의 결과를 z에 저장한 뒤

z를 출력하는 함수를 작성(함수의 이름은 f0로 하되, 매개변수를 받지 않는 형태)

```
> f0 <- function() { # 매개변수가 없는 함수 선언
```

```
+ x <- c(10, 20, 30, 40)
```

```
+ y <- c(1, 2, 3, 4)
```

```
+ z <- x * y
```

```
+ print(z) # z를 출력
```

```
+ }
```

```
>
```

```
> # 함수 호출
```

```
> f0()
```

```
[1] 10 40 90 160
```

- 위의 예에서 주의할 점은 함수 선언에 관련된 부분으로 매개변수가 없더라도

function()과 같이 반드시 괄호를 표시할 것

2. R 함수 만들기

예제 5-27 data와 num이라는 매개변수를 받아 data의 평균과 분산, 범위를 구하는 함수를 생성

(단, num이라는 매개변수에 따라 평균, 분산, 범위 중 한 가지 통계량만을 구하여 출력하되 기본값은 평균을 출력)

```
> f_default <- function(data, num=1) {    # "num=1"은 기본값으로 1
+ d.min <- min(data)
+ d.max <- max(data)
+ switch(num, mean(data), var(data), c(d.min, d.max)) # switch 함수
+ }
```

2. R 함수 만들기

▶ 함수를 다루는 함수

1) is.function()

is.function() 함수는 객체가 함수인지 아닌지 검증하는 함수로서 지정한 객체가 함수라고 판단되면 TRUE 값을, 함수가 아니면 FALSE 값을 반환

예제 5-28 [예제 5-27]과 같이 f_default라는 함수를 정의하고, 이렇게 생성된 f_default라는 객체가 함수인지를 판별

```
> f_default <- function(data, num=1) {  
+ d.min <- min(data)  
+ d.max <- max(data)  
+ switch(num, mean(data), var(data), c(d.min, d.max))  
+ }
```

```
> is.function(f_default)
```

```
[1] TRUE
```

- is.function(f_default) 명령문을 실행하였더니 TRUE 값이 반환되었으므로 f_default라는 객체는 함수

2. R 함수 만들기

2) args()

args() 함수는 함수의 매개변수들을 반환하는 함수

예제 5-29 [예제 5-27]의 f_default라는 함수의 매개변수로 무엇을 사용하는지 출력하기

```
> args(f_default)
```

```
function(data, num=1)
```

```
NULL
```

- data 매개변수와 num 매개변수를 갖고 있으며 num 매개변수는 기본값으로 1을 갖는다는 것을 알 수 있음

예제 5-30 R에 내장된 기본함수의 경우에도 args() 함수를 이용하면 매개변수를 파악할 수 있음

```
> args(log)
```

```
function(x, base=exp(1))
```

```
NULL
```

- x 매개변수와 base 매개변수를 갖고 있으며 base 매개변수의 기본값은 exp(1)의 값을 나타내고 있음

2. R 함수 만들기

3) attributes()

attributes() 함수는 함수의 소스 코드를 반환하는 역할을 하므로 R에서 사용자가 직접 생성한 함수가 인식되었다면 attributes() 함수를 통해 그 소스 코드를 확인해 볼 수 있음

예제 5-31 attributes() 함수를 이용하여 [예제 5-27]에 있는 f_default 함수의 소스 코드 출력하기

```
> attributes(f_default)
$srcref
function(data, num=1) {
  d.min <- min(data)
  d.max <- max(data)
  switch(num, mean(data), var(data), c(d.min, d.max))
}
```

- attribute() 함수를 실행하면 \$srcref라는 이름 아래에 저장되어 있던 소스 코드가 나타남

2. R 함수 만들기

▶ 함수 생성방법의 응용

1) 연산자의 생성

사용자의 정의에 따라 연산자를 생성하고 싶다면 %로 시작과 끝을 이루는 함수명을 갖는 함수를 생성

예제 5-32 두 변수값을 받아서 앞의 값에 2를 곱한 뒤 뒤쪽 값을 더하는 연산자를 생성

```
> "%a2b%" <- function(a,b) return(2*a+b)
```

```
> 3 %a2b% 5
```

```
[1] 11
```

- 주의할 점은 연산자 생성 시 반드시 인용부호와 % 표시를 병기해야 한다는 것임

2. R 함수 만들기

2) 함수 내에서의 함수 생성

함수는 객체이므로 함수 내에서 다른 함수를 정의할 수 있음

예제 5-33 함수 f 는 v 라는 지역변수에 2라는 값을 저장하고 이 함수 매개변수 y 값을 받은 뒤,
이 y 값과 v , 그리고 외부에서 지정되는 u 값을 더하여 제곱연산을 실시하는 함수

```
> f <- function(x) {
+ v <- 2
+ g <- function(y) return((u+v+y)^2) # 함수 내에서 함수를 정의
+ gu <- g(u)
+ print(gu)
+ }
> u <- 5
> f()
[1] 144
```

03

실습하기

10 강

다음시간안내 >>>

R그래픽스 기초 (1)